
CASE STUDY REPORT

TITLE

**Enterprise Information System: Designing a Complete Open-Source Software
Development Environment**

Jainish. Kakkad [92400103550]

Jay. Visana [92400103529]

Parth. Hirani [92400103541]

DEPARTMENT

COMPUTER ENGINEERING

ACADEMIC YEAR

2026-2027

SEMESTER

5TH

SUBJECT

OPEN SOURCE TECHNOLOGIES (01CE0526)

CLASS

EC 6

BATCH

B

CASE STUDY REPORT

RUBRICS FOR CASE STUDY

Sr. No.	Assessment Criteria	Excellent	Good	Satisfactory	Poor	Marks
1	Problem Analysis, Cloud Concepts & SDG Mapping	4 Marks: Clear problem identification, appropriate cloud concepts, and accurate SDG mapping.	3 Marks: Good analysis with minor gaps in concepts or SDG mapping.	1–2 Marks: Basic problem understanding with partial cloud concepts or SDG mapping.	0 Marks: Inadequate or incorrect analysis and no SDG mapping.	4
2	Security Analysis & Technical Depth	4 Marks: Comprehensive security analysis, well-defined architecture, appropriate technologies, and justified solutions.	3 Marks: Good technical explanation with minor omissions.	1–2 Marks: Limited security analysis and technical details.	0 Marks: No meaningful security analysis or technical explanation.	4
3	Report Quality & References	4 Marks: Well-structured report in IEEE format with comprehensive technical content, proper citations, and relevant references.	3 Marks: Good report structure with minor formatting or referencing issues.	1–2 Marks: Basic report with limited technical details or incomplete references.	0 Marks: Incomplete report with poor documentation and missing references.	4
	Total					12

Sr. No.	Case Study Title / Problem Statement	Cos	Date	R1(4)	R2(4)	R3(4)	Marks (12)	Signature
1	Enterprise Information System: Designing a Complete Open-Source Software Development Environment							

Ms. Priyanka Mangi
Subject Faculty

CASE STUDY REPORT

Enterprise Information System: Designing a Complete Open-Source Software Development Environment
OPEN SOURCE TECHNOLOGIES (01CE0526)

Co's Mapped - CO1: OSS concepts **CO2:** Version control **CO3:** Web application development **CO4:** Database technologies **CO5:** Testing, deployment & automation
SDG's Mapped - SDG 4: Quality Education **SDG 9:** Industry, Innovation and Infrastructure **SDG 17:** Partnerships for the Goals

I. Introduction

- A complete open-source development environment is proposed for a university Library Management System covering development, testing, deployment, security, and maintenance.
- OSS reduces proprietary licensing dependence, supports interoperability and self-managed deployment, and provides flexibility for university software systems.
- To select and justify suitable open-source tools for version control, development, database, deployment, testing, security, and automation.
- The study covers technology selection, architecture, CI/CD, testing, security, deployment, scalability, licensing, and cost considerations.

II. Organization/Application Background

- The proposed system is a web-based university Library Management System serving students, librarians, administrators, and university IT staff.
- The ecosystem includes a web application, database, web server, authentication, backups, monitoring, development tools, and deployment infrastructure.
- The system must provide catalog management, inventory, circulation, authentication, reporting, security, scalability, reliability, and maintainability.
- Open-source technologies reduce licensing dependence while enabling self-managed, interoperable, customizable, and cost-effective software infrastructure.

III. Problem Analysis

- The problem is to recommend a complete open-source technology stack that satisfies security, scalability, maintainability, interoperability, and deployment requirements.
- Challenges include tool compatibility, transactional data security, limited IT resources, increasing traffic, lifecycle integration, and comprehensive testing.
- The system requires secure authentication, scalability, efficient performance, reliable backups, modularity, portability, and controlled operational cost.
- Functional requirements include authentication, catalog, inventory, circulation, fines, reporting, and administration; non-functional requirements include security, scalability, reliability, performance, and maintainability.
- The system is web-based, uses mainly structured transactional data, must remain manageable by a small IT team, and should support future horizontal scaling.

CASE STUDY REPORT

IV. Open Source Solution Analysis

- GitLab CE, Git, Django, MySQL Community Edition, Nginx, Gunicorn, Docker Engine, pytest, Selenium, Bruno, JMeter, and OWASP ZAP are selected.
- Selections are based on open-source compliance, security, scalability, maintainability, interoperability, productivity, deployment simplicity, and cost.
- GitLab CE–MIT, Git–GPLv2+, Django–BSD, MySQL–GPL, Nginx–BSD, Docker Engine–Apache 2.0, Selenium–Apache 2.0, Bruno–MIT.
- The proposed stack reduces dependence on proprietary platforms while retaining equivalent lifecycle capabilities through self-managed open-source tools.
- OSS provides lower licensing dependence, self-managed control, customization, interoperability, community support, and long-term deployment flexibility.

V. System Architecture and Technology Stack

- The architecture uses a layered three-tier design connecting GitLab CI/CD, application services, database services, testing, security, and deployment.
- Debian Linux, GitLab CE, Git, Django, MySQL Community Edition, Nginx, Gunicorn, Docker, pytest, Bruno, Selenium, JMeter, and OWASP ZAP.
- GitLab Community Edition is selected for self-managed repositories, issues, merge requests, and integrated CI/CD.
- Django is selected for its ORM, authentication foundations, migrations, administration facilities, and suitability for structured data-driven systems.
- MySQL Community Edition is selected because the LMS contains highly structured and transactional relational data.
- Nginx is selected for HTTPS termination, reverse proxying, static content delivery, and future load balancing.
- Developers create branches, commit changes, run tests and security checks, build Docker images, deploy to staging, and promote approved releases to production.

VI. Database and Deployment Analysis

- The relational design contains users, books, copies, authors, categories, loans, returns, fines, and reporting relationships.
- CRUD operations support creating and maintaining catalog records, reading availability/history, updating circulation status, and retiring records under controlled policies.
- MySQL runs on a protected data tier separated from the public application layer and accessed through the Django application.
- The system is deployed on a university Linux server or VPS using Docker containers with Nginx, Gunicorn, Django, MySQL, backups, and monitoring.
- Nginx acts as the reverse proxy and can later distribute requests across multiple Django/Gunicorn application instances.

CASE STUDY REPORT

VII. Software Quality and Testing

- A complementary testing strategy combines unit, API, browser, performance, dependency, and dynamic security testing.
- pytest is used for unit and integration testing of business logic, models, authentication helpers, and database interactions.
- Bruno is selected for API testing because its version-controlled collections integrate naturally with Git; Postman remains a familiar syllabus reference.
- Selenium WebDriver is used for end-to-end browser testing of login, search, circulation, and reporting workflows.
- Apache JMeter is used to measure response time, throughput, concurrent-user behaviour, and peak-period performance.
- OWASP ZAP is used for dynamic security testing of the staging application to identify web-level vulnerabilities.
- Testing should verify expected functionality, response behaviour, performance, security findings, and successful CI/CD execution before release.

VIII. Open Source Community and Licensing Analysis

- The proposed environment combines MIT, GPL, BSD, and Apache 2.0 licensed open-source technologies according to each project's licensing terms.
- License notices, dependency licenses, compatibility, and distribution requirements must be reviewed and maintained throughout the project lifecycle.
- The selected projects provide public documentation, issue tracking, releases, contributor resources, and active open-source development ecosystems.
- Official documentation supports installation, configuration, development, security, troubleshooting, updates, and operational maintenance.
- Dependencies and runtime components should be reviewed periodically, with controlled upgrades, security patching, and reproducible release versions.
- Contributors can improve documentation, report issues, add tests, fix bugs, improve tooling, and submit upstream contributions where permitted.

IX. Technical Analysis

- Requirements are converted into development tasks, implemented on feature branches, validated through CI/CD, packaged into Docker images, deployed to staging, and promoted after approval.
- The solution provides an integrated open-source lifecycle, transactional data integrity, reproducible deployment, layered testing, security integration, and a practical scaling path.
- Self-managed infrastructure requires administration, the initial database can remain a single availability dependency, and automated security tools require human review.
- The system can progress from one application instance to multiple Django/Gunicorn instances, database scaling, caching, and eventually container orchestration.
- Reliability is supported through automated backups, health checks, versioned deployments, protected branches, staging, logging, and monitoring.

CASE STUDY REPORT

- Modular application design, documented deployment, controlled upgrades, repeatable environments, and CI/CD practices improve long-term maintainability.
- Performance is evaluated using JMeter metrics such as response time, throughput, concurrent users, and error rate, supported by efficient queries, indexing, caching, and horizontal scaling.

X. SDG Mapping and Justification

- SDG 4 – Quality Education, SDG 9 – Industry, Innovation and Infrastructure, and SDG 17 – Partnerships for the Goals.
- The solution supports digital education, open-source innovation, scalable infrastructure, and collaborative software development through interoperable technologies.
- A university LMS can improve access to library resources while the open-source development environment reduces licensing dependence and supports sustainable technical collaboration.

XI. Challenges and Recommendations

- Tool integration, database availability, testing complexity, container management, and increasing application load require structured engineering practices.
- Risks include broken access control, SQL injection, credential theft, insecure file handling, weak sessions, and vulnerable dependencies.
- Self-managed infrastructure requires secure configuration, backups, patching, monitoring, certificate management, and recovery procedures.
- Future work can introduce database replication, centralized monitoring, infrastructure-as-code, stronger security automation, and orchestration when justified.
- Start with a simple three-tier deployment, measure real requirements, automate testing and security, and introduce additional architectural complexity only when necessary.

XII. Conclusion

- The case study proposes a complete open-source development environment for a secure, scalable, and maintainable university Library Management System.
- GitLab CE, Django, MySQL, Nginx, Gunicorn, Docker, pytest, Selenium, Bruno, JMeter, and OWASP ZAP provide an integrated lifecycle-oriented technology stack.
- The study demonstrates requirements analysis, technology selection, open-source licensing, architecture design, database selection, testing, CI/CD, security, and deployment planning.
- The proposed approach reflects practical industry practices such as DevOps, DevSecOps, containerization, CI/CD, version control, automated testing, and scalable architecture.

CASE STUDY REPORT

Building a Complete Open-Source Development Environment

Jainish Kakkad¹
Department of Computer
Engineering
Marwadi University
Rajkot, India
jainishkakkad.133582@marwadiuniversity.ac.in

Jay Visana²
Department of Computer
Engineering
Marwadi University
Rajkot, India
jay.visana133227@marwadiuniversity.ac.in

Parth Hirani³
Department of Computer
Engineering
Marwadi University
Rajkot, India
parth.hirani133461@marwadiuniversity.ac.in

Abstract— This case study develops a requirements-driven, fully open-source development environment for a university Library Management System. The design covers the complete software lifecycle: version control, application development, data management, testing, security, deployment, automation, and maintenance. A structured technology-selection method is applied to the library requirements using criteria such as security, scalability, maintainability, interoperability, automation, deployment simplicity, and cost. The proposed environment uses self-managed GitLab Community Edition for source control and CI/CD, Django for application development, MySQL Community Edition for transactional data, Nginx and Gunicorn for application delivery, Docker Engine for reproducible packaging, and pytest, Selenium, Bruno, Apache JMeter, and OWASP ZAP for complementary quality and security activities. The deployment follows a staged three-tier architecture that can scale horizontally when demand justifies it. The resulting design addresses the stated open-source constraint while keeping operational complexity proportionate to the university library use case.

Keywords— *open-source software, DevOps, Library Management System, GitLab, Django, MySQL, CI/CD, software testing, DevSecOps.*

CASE STUDY REPORT

I. INTRODUCTION

A. Background of the Case Study

The assigned case study asks for a complete open-source development environment for a university Library Management System (LMS). The proposed environment must cover version control, application development, data management, testing, security, deployment, automation, and maintenance rather than treating each tool as an isolated choice. The library domain is a suitable engineering context because it combines structured records, user roles, circulation transactions, reporting, and long-term operational requirements.

B. Importance of Open Source Technologies

Open Source Software (OSS) can reduce dependence on proprietary licensing, enable self-managed deployment, support interoperability across the software lifecycle, and provide access to source code and community-driven documentation. In this case, the open-source requirement is not interpreted as zero cost: infrastructure, network connectivity, backup storage, electricity, and human administration remain real operational expenses [1], [9], [11], [13].

C. Objectives

- Select a Git platform that can support collaborative source control, issues, merge requests, and CI/CD.
- Recommend an open-source web framework suitable for a data-driven university system.
- Select a database for structured and transactional LMS data.
- Design a practical deployment architecture using open-source server and container technologies.
- Define complementary unit, API, browser, performance, and security testing tools.
- Justify technology choices using research evidence, official documentation, requirements, and engineering trade-offs.

D. Scope of the Study

The study includes architecture recommendation, technology comparison, repository workflow, CI/CD workflow, testing strategy, security controls, deployment design, scalability path, open-source licensing, and cost considerations. It does not include production implementation of the complete LMS, certification for a specific university population, or a claim that one stack is universally superior. Kubernetes, a service mesh, managed proprietary CI/CD services, and proprietary SaaS testing platforms are not mandatory components of the initial reference architecture.

II. ORGANIZATION/APPLICATION BACKGROUND

A. Application Overview

The target application is a web-based Library Management System serving students, librarians, administrators, and the university IT team. The system manages users, books, physical copies, loans, returns, fines, reports, and administrative settings. A small software team develops and maintains the system collaboratively, while the university IT team is responsible for deployment, security, backups, monitoring, and operational maintenance.

B. Existing Software Ecosystem

A representative university-library ecosystem contains a web application, a relational data store, a web server, identity and access controls, backup storage, monitoring, and developer tooling. Research on higher-education library information systems identifies cataloging, inventory, authentication, circulation, search, and reporting as central functions and describes layered web deployments in higher-education settings [3].

C. Business Requirements

TABLE I. STAKEHOLDER REQUIREMENTS

Stakeholder	Primary needs
Students	Search catalog, view availability, borrow/renew/return status, history, and overdue information.
Librarians	Manage catalog, inventory, circulation, overdue records, and reports.
Administrators	Manage users, roles, system settings, audit information, and operational reporting.
University IT	Deploy, secure, back up, monitor, and maintain the system.
Software team	Develop, test, review, release, and maintain the software collaboratively.

D. Need for Open Source Technologies

The client requires a cost-effective, scalable, secure, and fully open-source solution. OSS supports this requirement by allowing the central development and deployment components to be self-managed and integrated. A requirements-driven selection approach is therefore more appropriate than selecting tools based only on popularity or isolated feature counts [1], [2].

III. PROBLEM ANALYSIS

A. Problem Identification

The engineering problem is to recommend a connected technology stack that supports the LMS lifecycle while satisfying the constraints of open-source licensing, security, maintainability, scalability, interoperability, deployment simplicity, and cost. The challenge is not only to name tools, but to show how they work together from requirements through release and maintenance.

B. Existing Challenges

- Different lifecycle tools may use incompatible workflows or licensing models.

CASE STUDY REPORT

- Library records require transactional consistency, role-based authorization, and controlled access.
- A small university IT team needs an operationally understandable deployment model.
- Growth in user traffic can expose bottlenecks if the application and database tiers are tightly coupled.
- Security feedback must be integrated into development rather than delayed until the end of a release.
- Testing must cover different failure modes; no single tool provides unit, API, browser, performance, and dynamic security coverage.

C. Engineering Requirements

TABLE II. NON-FUNCTIONAL REQUIREMENTS

Property	Engineering implication
Security	Authentication, authorization, TLS, safe secret handling, dependency scanning, and dynamic security testing.
Scalability	Stateless application tier where practical, reverse proxying, and ability to add application instances.
Performance	Efficient queries, indexing, caching where justified, and repeatable load testing.
Reliability	Transactional database operations, backups, health checks, and controlled releases.
Maintainability	Modular application code, documented deployment, and repeatable environments.
Portability	Containerized packaging and infrastructure-as-code-ready configuration.
Cost-effectiveness	Open-source software preference and avoidance of unnecessary operational complexity.

D. Functional Requirements

TABLE III. FUNCTIONAL REQUIREMENTS

Module	Key functions
Authentication and authorization	Login, logout, role-based access control, password management, and session control.
Catalog management	Books, authors, categories, publication information, and search.
Inventory management	Physical copies, availability, condition, and stock status.
Circulation	Issue, renewal, return, due dates, and transaction history.
Fine management	Overdue calculation, fine status, and settlement records.

Module	Key functions
Reporting	Circulation statistics, overdue reports, usage summaries, and administrative reports.
Administration	User management, roles, parameters, audit, and operational settings.

E. Constraints and Assumptions

- The LMS is accessed through a web application by students, librarians, and administrators.
- Dominant data is structured and transactional: users, books, copies, loans, returns, fines, and reports.
- The initial system should be manageable by a small university IT team.
- The architecture should have a practical path to horizontal scaling.
- Core software should be open-source and suitable for self-managed deployment.

IV. OPEN SOURCE SOLUTION ANALYSIS

A. Research-Based Technology Selection Method

The central methodological decision is to treat technology selection as an architecture problem. DevOps-stack research describes a sequence beginning with business-problem identification and requirements, followed by reference architecture, risk identification, continuous integration, continuous delivery, infrastructure automation, observability, and pipeline management [1]. ATA independently frames stack selection around user requirements and interoperability across architectural layers [2].

The research-to-architecture process used in this case is summarized in Fig. 1. Requirements are translated into selection criteria, open-source alternatives are compared, and the selected stack is then validated through lifecycle activities such as CI/CD and testing.

Requirements-driven technology selection

Requirements → criteria → alternatives → compatible stack → lifecycle validation

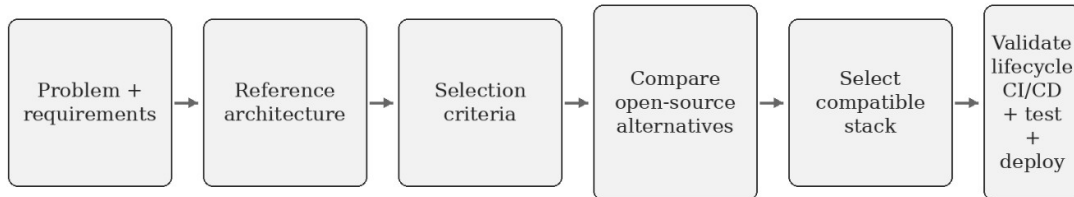


Fig. 1. Requirements-to-architecture technology selection process.

CASE STUDY REPORT

B. Selection Criteria

TABLE IV. TECHNOLOGY SELECTION CRITERIA

Criterion	Weight	Reason
Open-source compliance	20%	Mandatory constraint from the assignment.
Security	15%	Library systems contain user identities and borrowing records.
Maintainability	15%	University systems are expected to operate for years.
Scalability	15%	Demand can grow across academic periods.
Interoperability	10%	Tools must integrate across the lifecycle.
Development productivity	10%	The team must deliver features efficiently.
Deployment simplicity	10%	Small IT teams should be able to operate the system.
Cost	5%	The environment must be cost-effective.

C. Evidence Model and Research Synthesis

- Peer-reviewed research is used for selection criteria, observed trade-offs, and lifecycle practices.
- Official project documentation is used for current licensing and product capabilities.
- Case-specific reasoning connects those facts to the university-library context.
- Where studies use different populations or disagree, the report preserves the context rather than treating one finding as universal.

TABLE V. RESEARCH SYNTHESIS

Research area	Contribution
DevOps stack definition	Structured requirements-to-stack process and lifecycle automation.
Architecture-based technology selection	Complete-stack interoperability rather than isolated tool choice.
Open-source DevOps integration	Evidence that integrated OSS toolchains can address interconnection and CI/CD issues.

Research area	Contribution
Git platform evaluation	Multi-criteria selection of source-code-management tooling.
Framework comparison	Security, scalability, maintainability, and performance dimensions.
Database architecture	Trade-offs between relational integrity and document-oriented flexibility.
Automated testing	Different tools serve UI, validation, and performance roles.
DevSecOps	Security feedback should be inserted into the CI/CD lifecycle.
Library-system studies	Concrete higher-education requirements and layered deployment context.

D. Selected Open Source Technologies and Benefits

The proposed core stack is selected as an integrated environment rather than a collection of unrelated tools. Git provides distributed source control; GitLab Community Edition provides a self-managed collaboration and CI/CD platform; Django provides the application framework; MySQL Community Edition provides transactional relational storage; Nginx and Unicorn provide application delivery; Docker Engine provides reproducible packaging; and pytest, Selenium, Bruno, Apache JMeter, and OWASP ZAP address complementary quality and security activities. The licensing basis of these components is documented in Section VIII.

V. SYSTEM ARCHITECTURE AND TECHNOLOGY STACK

A. Overall Architecture

The proposed environment follows a layered lifecycle: development and version control feed automated quality and security checks; a reproducible container image is promoted to staging; the runtime consists of Nginx, Unicorn, Django, and MySQL; and backups plus monitoring support operation. The proposed stack and lifecycle integration are shown in Fig. 2.

Proposed open-source environment

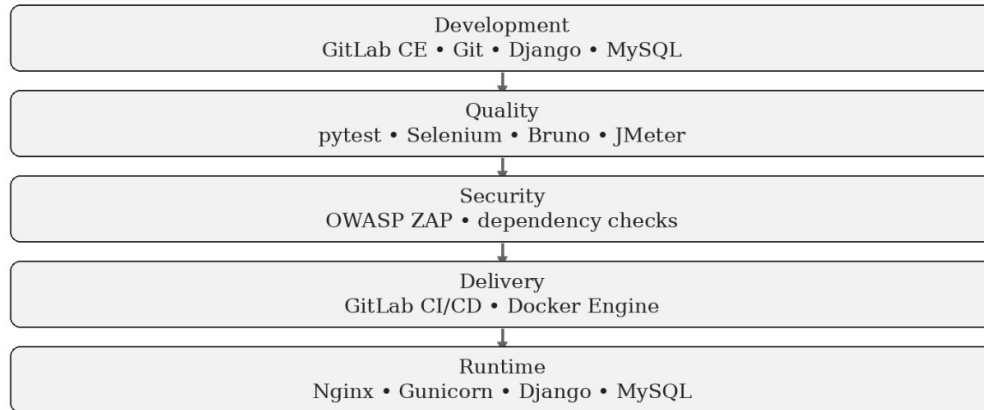


Fig. 2. Proposed open-source technology stack and lifecycle integration.

CASE STUDY REPORT

TABLE VI. PROPOSED TECHNOLOGY STACK

Layer / role	Selected technology	Primary responsibility
OS / Server	Debian Linux	Server foundation.
Version control	Git	Distributed source control.
Git platform	GitLab Community Edition	Repositories, issues, merge requests, and CI/CD.
Web framework	Django	Application logic, routing, ORM, authentication foundations.
Database	MySQL Community Edition	Structured transactional data.
Web server	Nginx	TLS termination, reverse proxying, static content, future load balancing.
Application server	Gunicorn	Serve the Django WSGI application.
Containerization	Docker Engine	Reproducible packaging and deployment.
CI/CD	GitLab CI/CD + Runner	Build, test, security checks, and deployment.
Unit tests	pytest	Unit and integration testing.
Browser tests	Selenium WebDriver	End-to-end browser regression tests.
API tests	Bruno	Version-controlled API collections and automated validation.
Performance	Apache JMeter	Load and performance testing.
Security	OWASP ZAP	Dynamic application security testing.

B. Version Control and Repository Policy

Git is used for distributed source control. The repository policy keeps main as the protected production-ready branch, uses develop as an integration branch, and isolates feature/*, bugfix/*, and hotfix/* work. All changes enter main through a reviewed merge request and successful required pipeline checks. Conventional commit prefixes include feat, fix, docs, refactor, test, chore, and security.

C. Git Platform Selection

GitLab Community Edition is selected for the reference environment because it can combine self-managed repository management, issue tracking, merge requests, and CI/CD within one lifecycle. Research comparing GitHub Actions and GitLab CI/CD in an educational setting found no statistically significant difference in technology acceptance in a 21-participant study, while another multi-criteria evaluation treated source-code-management tooling as an engineering selection problem [4], [5]. The selection is therefore driven primarily by the assignment's self-managed fully open-source requirement rather than by a universal ranking of platforms.

TABLE VII. GIT PLATFORM ALTERNATIVES

Platform	Relevant strengths	Case-study concern
GitLab CE	Self-managed repositories, merge requests, issues, and integrated CI/CD.	Requires university administration of the GitLab instance.
GitHub + Actions	Strong ecosystem and CI/CD; widely used in open-source development.	Hosted platform and self-hosted OSS licensing models are not identical.
Bitbucket	Repository collaboration and CI/CD integrations.	Less aligned with the strict self-managed open-source objective.

D. Web Framework Selection

Laravel, Django, and the Node.js ecosystem are credible candidates. The 2025 IEEE ERP study discusses effects of framework choice on scalability, security, maintainability, and development speed, while a 2026 Python framework comparison evaluates architecture, performance, scalability, maintainability, and interoperability [6], [7]. Django is selected because the LMS is structured and data-driven, and Django provides an ORM, authentication foundations, migrations, administration facilities, and a mature Python ecosystem [10]. Laravel remains a credible alternative, particularly for teams already centered on PHP.

TABLE VIII. WEB FRAMEWORK CANDIDATES

Candidate	Relevant characteristics	Case fit
Laravel	PHP MVC framework; mature ecosystem and rapid web development.	Strong alternative for PHP-centered teams.
Django	Python framework with ORM, authentication foundations, migrations, and admin facilities.	Strong fit for structured, data-driven institutional systems.
Node.js ecosystem	Asynchronous JavaScript runtime paired with frameworks such as Express or Fastify.	Strong for API-driven and real-time workloads.

VI. DATABASE AND DEPLOYMENT ANALYSIS

A. Database Design and CRUD Operations

The LMS core has strong relationships and transactional dependencies. A loan references a user and a physical book copy; a copy references a title; a title can have authors and categories; overdue status derives from transaction dates; and reports aggregate transaction data. These relationships map naturally to a relational data model. CRUD operations therefore include creating and maintaining catalog records, reading availability and history, updating circulation state, and deleting or retiring administrative records according to policy.

B. Database Selection

TABLE IX. DATABASE CANDIDATES

Candidate	Strengths	Case fit
MySQL Community Edition	Relational schema, transactions, joins, constraints, mature tooling, and replication options.	Strong fit for core LMS transactions.
PostgreSQL	Advanced SQL, extensibility, rich data types, and strong transactional capabilities.	Technically strong alternative if preferred by the university.
MongoDB	Document model and flexible schema for evolving or unstructured data.	Useful for specialized modules but not necessary as the primary store.

The 2026 IEEE database study compares MySQL and MongoDB across monolithic and microservices architectures and identifies trade-offs involving modularity, scalability, extensibility, structured modelling, and transactional integrity [8]. MySQL Community Edition is selected as the primary database because the core LMS workload is relational and transaction-heavy [11].

CASE STUDY REPORT

MongoDB is retained as a future option for specialized document-like metadata where a flexible schema is justified.

C. Runtime Deployment Architecture

The proposed deployment uses a university-owned Linux server or a VPS, with Nginx at the public edge, Django/Gunicorn on a protected application tier, MySQL on an internal data tier, separate backup storage, and logging/monitoring. The runtime deployment architecture is shown in Fig. 3.

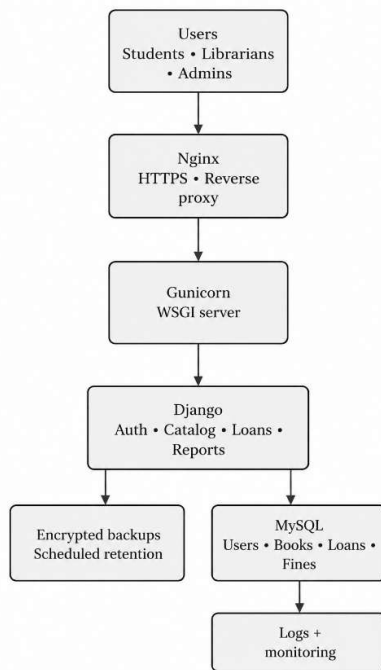


Fig. 3. Runtime deployment architecture with staged horizontal scaling.

TABLE X. RUNTIME DEPLOYMENT LAYERS

Layer	Component	Responsibility
Edge	Nginx	HTTPS/TLS termination, reverse proxying, static assets, and future request distribution.
Application server	Gunicorn	Runs the Django WSGI application with multiple worker processes.
Application	Django	Authentication, authorization, catalog, circulation, reports, and business logic.
Data	MySQL	Persistent transactional data and relational constraints.
Persistence support	Backup store	Scheduled encrypted backups and recovery copies.
Operations	Logs/monitoring	Health information for diagnosis and maintenance.

D. Cloud and Deployment Concepts

The reference architecture can be deployed using Infrastructure as a Service (IaaS) through a university VPS or a university-owned virtualized server. Containerization

with Docker Engine provides a repeatable application package, while network segmentation keeps the database and application tiers away from direct public exposure. Separate backup storage provides recovery independence from the primary host. Horizontal scaling is introduced only when demand or availability requirements justify additional application instances.

E. Why Nginx and Docker?

Nginx is used as an open-source HTTP server and reverse proxy and can later participate in load balancing [12]. Docker Engine packages the application and runtime dependencies into reproducible containers and is used here rather than assuming Docker Desktop is required for production [13].

F. Why Not Kubernetes at the Start?

Kubernetes is valuable for larger multi-service environments, but the case does not specify a scale that requires cluster orchestration. Starting with a small number

of Docker containers behind Nginx keeps the operational model understandable and lowers administrative burden. Migration to Kubernetes remains possible if traffic, service count, or high-availability requirements justify it.

VII. SOFTWARE QUALITY AND TESTING

A. Testing Strategy

The testing strategy is complementary. The 2025 IEEE automated-testing study compares Selenium WebDriver, Cypress, and Apache JMeter and demonstrates that browser automation and performance testing address different classes of quality problem [14]. The LMS therefore uses separate layers for unit/integration logic, API behaviour, end-to-end browser workflows, performance, dependency/quality checks, and dynamic security.

TABLE XI. TESTING LAYERS

Test layer	Tool	Example LMS tests	Pipeline stage
Unit / integration	Pytest	Loan rules, fine calculation, authentication helpers, model logic, database integration.	Every merge request
API	Bruno	Login, catalog search, circulation endpoints, authorization failures.	Every merge request
End-to-end browser	Selenium WebDriver	Student login, search, librarian issue/return, reporting.	Merge request / release
Performance	Apache JMeter	Concurrent search, login, circulation, peak-period traffic.	Scheduled / release
Dynamic security	OWASP ZAP	Passive and active web-security checks against staging.	Release / scheduled
Dependency / quality	Dependency-Check + quality tooling	Known dependency vulnerabilities and maintainability checks.	Every merge request

B. Test Data and Evidence

- Use synthetic students, books, and transactions; never use real personal data in tests.
- Store test definitions in version control.

CASE STUDY REPORT

- Generate JUnit-style test results and pipeline artifacts.
- Record response time, throughput, error rate, and failed scenarios for performance experiments.
- Record ZAP findings with severity, affected endpoint, reproduction path, and mitigation status.

C. CI/CD and Automation

GitLab CI/CD is used to connect validation, testing, security, packaging, staging deployment, and production promotion. GitLab documents CI/CD as a build-test-deploy-monitor process driven by .gitlab-ci.yml and executed by runners; self-managed runners can execute jobs on university-controlled infrastructure [17]. The proposed workflow is shown in Fig. 4.

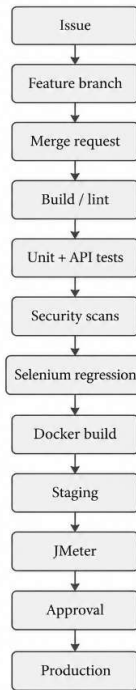


Fig. 4. Proposed GitLab CI/CD workflow for controlled deployment.

TABLE XII. PROPOSED PIPELINE STAGES

Stage	Automated activity	Purpose
Validate	Linting, formatting, dependency checks	Catch basic quality and dependency problems early.
Test	pytest + Bruno CLI tests	Verify business logic and API behaviour.
Security	Dependency/SAST-style checks + ZAP where appropriate	Move security feedback earlier in the lifecycle.
Browser	Selenium regression suite	Validate critical end-user workflows.
Package	Docker image build and tag	Create a reproducible release artifact.
Staging	Deploy to staging	Validate deployment in a production-like environment.

Stage	Automated activity	Purpose
Performance	JMeter scheduled/release test	Check load and response-time behaviour.
Approval	Human release gate	Provide controlled production promotion.
Production	Deploy approved image	Release repeatably with minimal manual intervention.

D. CI/CD Policy

- Every merge request must pass unit and API tests.
- Security checks are required before release.
- Production deployment requires approval for the protected branch.
- Secrets are injected through protected variables or a dedicated secret-management system; credentials are never committed to Git.
- The same versioned Docker image promoted through staging should be promoted to production.

Research on continuous-deployment maturity indicates that compatible technology stacks and repeatable processes support progress toward continuous delivery, while maintaining maturity requires sustained attention to process quality [18]. Open-source DevSecOps research similarly supports moving security analysis into the normal pipeline rather than separating it from development [19], [20].

VIII. OPEN SOURCE COMMUNITY AND LICENSING ANALYSIS

A. Licensing Model

TABLE XIII. LICENSING AND COMPLIANCE

Component	License / status	Compliance consideration
GitLab CE	MIT	Use the documented self-managed Community Edition license [9].
Git	GPLv2+	Open-source distributed version-control project.
Django	BSD	Open-source framework licensed under the BSD license [10].
MySQL Community Edition	GPL	Open-source Community distribution [11].
Nginx	2-clause BSD	Open-source web server and reverse proxy [12].
Docker Engine	Apache 2.0	Open-source container engine [13].
Selenium	Apache 2.0	Open-source browser automation project [15].
Bruno	MIT	Open-source Git-friendly API client [16].
JMeter / OWASP ZAP / pytest	Apache 2.0 / Apache 2.0 / MIT	Open-source testing and security projects.

B. License Compliance

License compliance requires retaining applicable copyright and license notices, reviewing transitive dependencies, checking compatibility before distribution, and documenting any license exceptions. The phrase “fully open-source” in this case means that the proposed core environment can be built and operated with open-source components subject to dependency review; it does not mean that all infrastructure or administration becomes free.

CASE STUDY REPORT

C. Community Support and Documentation

The selected projects are maintained through public documentation, issue tracking, release/version management, and contributor-oriented workflows. Practical contribution opportunities include documentation improvements, defect reports, test additions, bug fixes, and upstream contributions where project policies permit. Repository-level contribution guidance should mirror the conventions of the project and avoid undocumented local practices.

D. Version Management

Version management is treated as an engineering responsibility. Dependencies and runtime components should be reviewed periodically, security patches should be applied in a controlled cycle, and release artifacts should remain reproducible. The CI/CD pipeline provides a repeatable path for validating new versions before production promotion.

IX. TECHNICAL ANALYSIS

A. Working of the Proposed Solution

A university requirement is translated into an issue or feature specification. A developer creates a branch, implements the Django application against MySQL, and submits a merge request. CI/CD validates the change using unit and API tests, security and dependency checks, browser regression, and packaging. The approved container image is deployed to staging, performance validation is executed as planned, and a controlled release promotes the same artifact behind Nginx and Gunicorn. Logs, backups, and monitoring support maintenance.

B. Advantages

- The lifecycle is connected through one coherent open-source toolchain.
- Relational modelling supports transactional integrity for circulation data.
- Containerized packaging improves reproducibility between development, staging, and production.
- Layered testing provides broader quality coverage than a single testing tool.
- Security checks are moved earlier into the development and release lifecycle.
- The architecture can scale the application tier without requiring microservices at the first release.

C. Limitations

- Self-managed infrastructure creates administration and maintenance responsibilities.
- The initial deployment may have a single-database availability dependency.
- Automated security checks can produce false positives and false negatives and require human triage [19].
- Extensive browser testing can increase pipeline duration.

- The choice of staged architecture does not eliminate the need for future capacity testing.

D. Scalability

TABLE XIV. STAGED SCALING PATH

Stage	Architecture	When justified
Stage 1	One Nginx + one application host + MySQL	Initial deployment and moderate load.
Stage 2	Multiple Django/Gunicorn containers behind Nginx	Application CPU or concurrency becomes the bottleneck.
Stage 3	Database replication/read scaling + cache	Read-heavy workloads or stronger availability needs increase.
Stage 4	Container orchestration	Multiple services, high availability, and operational automation justify orchestration.

E. Reliability

- Automated backups with tested restoration procedures.
- Health checks for application and database services.
- Versioned deployment artifacts.
- Protected production branch and controlled approvals.
- Staging environment before production.
- Centralized logging and basic system monitoring.

F. Maintainability

The architecture separates application logic, data access, delivery, testing, security checks, and deployment configuration. Documentation is maintained alongside source code so that operational knowledge does not depend only on individual team members. A monthly dependency and security review, periodic database index and backup review, JMeter checks around high-traffic periods, and periodic recovery tests keep the environment maintainable [18].

G. Performance Analysis

Performance should be evaluated using repeatable measurements rather than assumptions. JMeter can measure response time, throughput, concurrent-user behaviour, and error rate at the HTTP layer [14]. Efficient SQL queries, indexing, caching where justified, and horizontal application scaling provide the primary performance path. The architecture intentionally avoids premature microservices because distribution adds network hops and operational overhead without a stated requirement.

CASE STUDY REPORT

X. SDG MAPPING AND JUSTIFICATION

TABLE XV. SDG MAPPING

SDG	Connection to the case
SDG 4 – Quality Education	A digital library platform improves structured access to educational resources. The open-source development environment also provides a practical context for learning software engineering, version control, testing, and DevOps practices.
SDG 9 – Industry, Innovation and Infrastructure	The solution applies open-source engineering, automation, layered architecture, containerization, testing, and scalable infrastructure practices to a real institutional software scenario.
SDG 17 – Partnerships for the Goals	Collaborative development is supported through source control, issue tracking, merge requests, CI/CD, contribution documentation, and interoperable open-source tools.

XI. CHALLENGES AND RECOMMENDATIONS

A. Technical, Security, and Deployment Risks

TABLE XVI. RISKS AND MITIGATIONS

Risk / challenge	Impact	Recommended mitigation
Self-managed GitLab administration	Platform outage can affect collaboration.	Backups, documented recovery, update policy, and separate runner infrastructure.
Single database dependency	Database outage affects circulation operations.	Scheduled backups, tested restoration, and a future replication option.
Automated security false positives	Developer time may be spent triaging noise.	Triage policy, justified suppression, and periodic rule review.
Growing browser-test suite	Long UI suites can slow releases.	Keep critical-path Selenium tests small and use unit/API tests for broader coverage.
Docker operational complexity	Improper container management can add overhead.	Use a small number of documented containers and standard images.
Overengineering	Unnecessary microservices/Kubernetes increase operational burden.	Use staged architecture and require a measurable trigger for added complexity.

B. Cost and Open-Source Compliance

TABLE XVII. OPERATIONAL COST CATEGORIES

Cost category	Example
Compute	University server or VPS.
Storage	Primary disk plus backup storage.
Networking	Internet connectivity, DNS, and firewalling.
Operations	IT administration and security patching.
Recovery	Off-site backup or secondary server.
Training	Developer and administrator learning time.

Open-source licensing can remove mandatory software licence fees for the core stack, but infrastructure and people remain costs. The final selection therefore optimizes not only software licensing but also operational complexity and maintenance effort [9], [13].

C. Future Improvements

- Introduce database replication when availability or read volume justifies it.
- Add centralized observability and alerting as operational usage grows.
- Expand automated security analysis and dependency monitoring.
- Add infrastructure-as-code for repeatable server and environment provisioning.
- Move to stronger orchestration only when multiple services or availability requirements justify it.
- Introduce formal disaster-recovery objectives and test them periodically.

D. Recommendation for Better Implementation

The implementation should preserve a simple first release, measure actual operational needs, and add architectural complexity only when a measurable requirement appears. The e-Archive architecture research demonstrates that more distributed architectures can be justified when large-scale data, horizontal scaling, or interoperability create specific demands [22]. The LMS should therefore use a staged scaling path rather than assuming that the largest possible architecture is automatically appropriate.

XII. CONCLUSION

Case Study 15 is best treated as an integrated architecture and technology-selection exercise rather than as six disconnected tool descriptions. The proposed environment translates the case requirements into a coherent lifecycle covering source control, development, data management, testing, security, deployment, automation, and maintenance. GitLab Community Edition provides the self-managed collaboration and CI/CD platform; Django supports the structured web application; MySQL Community Edition supports transactional data; Nginx and Gunicorn provide application delivery; Docker Engine provides reproducible packaging; and pytest, Selenium, Bruno, JMeter, and OWASP ZAP provide complementary quality and security activities. The architecture deliberately avoids unnecessary microservices and Kubernetes in the initial release and instead provides a staged scaling path. This keeps the operational burden proportionate to the stated problem while retaining technical growth options.

CASE STUDY REPORT

REFERENCES

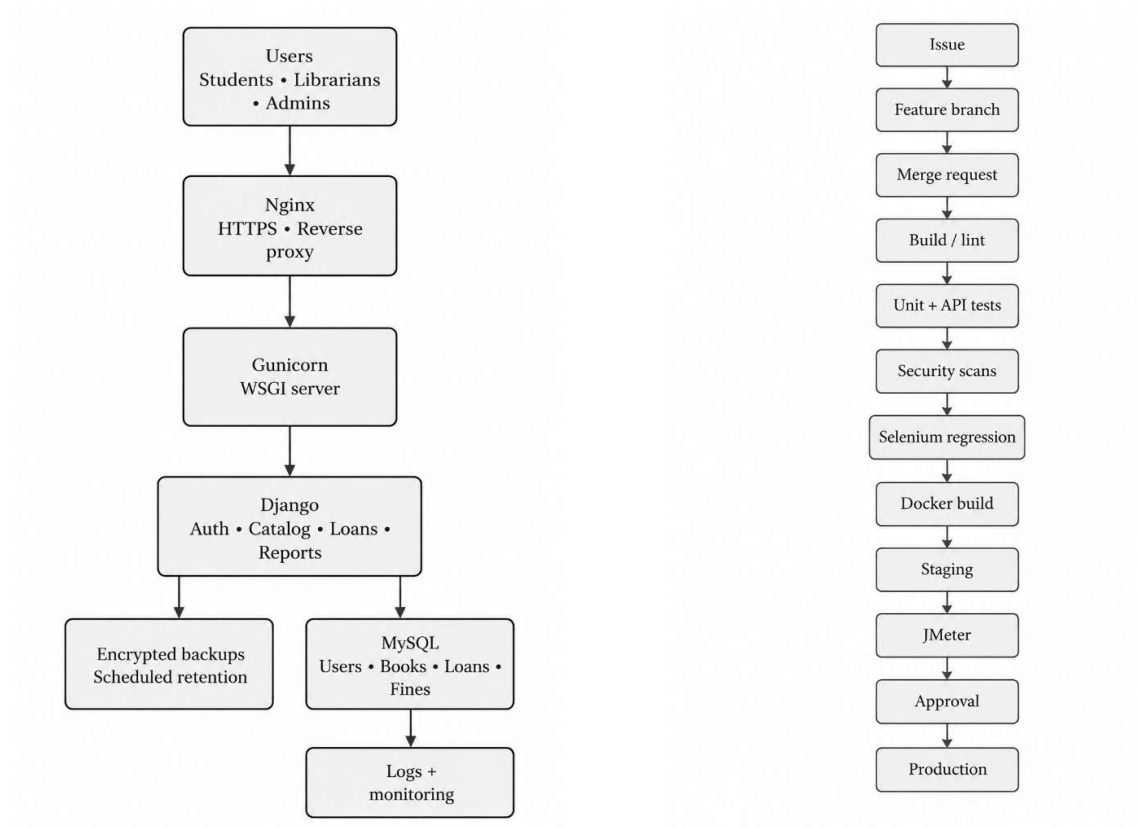
- [1] G. Marquez and H. Astudillo, "Framework to support students in defining DevOps technology stacks," in Proc. 51st Latin American Computer Conference (CLEI), 2025, pp. 1–10, doi: 10.1109/CLEI67442.2025.11420366.
- [2] C. Phalak and S. Kunde, "ATA: Architecture-based technology advisor tool," in Proc. ACM International Conference, 2018, doi: 10.1145/3241403.3241408.
- [3] B. Biibosunov, N. Zholochubekov, K. Begaly, K. Achekeev, and K. Mukudin uulu, "Design and implementation of a web-based library information system for higher education institutions," in Proc. 2025 Asian Conference on Communication and Networks (ASIANComNet), 2025.
- [4] A. Rodriguez and B. Pando, "Acceptance of CI/CD tools in education: A comparison of GitHub Actions and GitLab," in Proc. 13th Int. Conf. Software Engineering Research and Innovation (CONISOFT), 2025, doi: 10.1109/CONISOFT66928.2025.00034.
- [5] P. Mittal and P. Narang, "Performance assessment and analysis of development and operations based automation tools for source code management," Int. J. Electrical and Computer Engineering, vol. 13, no. 2, pp. 1817–1826, 2023, doi: 10.11591/ijece.v13i2.pp1817-1826.
- [6] S. Miftari et al., "Client-server architecture for ERP systems: A comparative analysis of Laravel, Django and ASP.NET Core," in Proc. IEEE MIPRO, 2025, doi: 10.1109/MIPRO65660.2025.11132028.
- [7] I. Ahmed, P. Arora, N. Taneja, S. P. Sharma, and M. Nazim, "A comparative evaluation of modern Python full-stack frameworks: Architecture, performance, and scalability," in Proc. IEEE ICICI, 2026, doi: 10.1109/ICICI68867.2026.11564850.
- [8] A. Merić and B. Isaković, "From relational to document-based: How database selection shapes software architecture in distributed systems," in Proc. 30th Int. Conf. Information Technology (IT), 2026, doi: 10.1109/IT67293.2026.11435700.
- [9] GitLab, "GitLab licensing and compatibility," official documentation, accessed Sep. 22, 2026. [Online]. Available: <https://docs.gitlab.com/development/licensing/>
- [10] Django Software Foundation, "How to get Django," official documentation, accessed Sep. 22, 2026. [Online]. Available: <https://www.djangoproject.com/download/>
- [11] Oracle MySQL, "MySQL Community Edition," official documentation, accessed Sep. 22, 2026. [Online]. Available: <https://www.mysql.com/products/community/>
- [12] NGINX, "nginx," official project documentation, accessed Sep. 22, 2026. [Online]. Available: <https://nginx.org/>
- [13] Docker, "Docker Engine," official documentation, accessed Sep. 22, 2026. [Online]. Available: <https://docs.docker.com/engine/>
- [14] Z. M. Azmi, S. Rochimah, D. D. P. Baron, and I. Fadhil, "Performance analysis of automated testing tools selection: A case study of supporting application in construction company," in Proc. IEEE ICISS, 2025, doi: 10.1109/ICISS66954.2025.11389536.
- [15] Selenium, "Copyright and attributions," official documentation, accessed Sep. 22, 2026. [Online]. Available: <https://www.selenium.dev/documentation/about/copyright/>
- [16] Bruno, "API client," official project documentation, accessed Sep. 22, 2026. [Online]. Available: <https://www.usebruno.com/product/api-client>
- [17] GitLab, "Get started with GitLab CI/CD," official documentation, accessed Sep. 22, 2026. [Online]. Available: <https://docs.gitlab.com/ci/>
- [18] S. Mäkinen, M. Puonti, T. Lehtonen, T. Mikkonen, T. Kilamo, and T. Männistö, "Revisiting continuous deployment maturity: A two-year perspective," in Proc. ACM Symposium on Applied Computing, 2019, pp. 1810–1817, doi: 10.1145/3297280.3297458.
- [19] C. Feio, N. Santos, N. Escravana, and B. Pacheco, "An empirical study of DevSecOps focused on continuous security testing," in Proc. IEEE European Symposium on Security and Privacy Workshops (EuroS&PW), 2024, doi: 10.1109/EuroSPW61312.2024.00074.
- [20] W.-T. Lee and Z.-W. Liu, "Microservices-based DevSecOps platform using pipeline and open source software," Journal of Information Science and Engineering, vol. 39, no. 5, pp. 1117–1128, 2023, doi: 10.6688/JISE.202309_39(5).0007.
- [21] Y. Yulianto, E. Utami, and Kusnawi, "Automatic deployment pipeline for containerized application of IoT device," in Proc. 6th Int. Conf. Information Technology, Information Systems and Electrical Engineering (ICITISEE), 2022, pp. 81–86, doi: 10.1109/ICITISEE57756.2022.10057699.
- [22] H. Falatiuk, M. Shirokopetleva, and Z. Dudar, "Investigation of architecture and technology stack for e-Archive system," in Proc. IEEE PIC S&T, 2019, doi: 10.1109/PICST47496.2019.9061407.

CASE STUDY REPORT

APPENDIX A. REQUIREMENT-TO-TECHNOLOGY TRACEABILITY

TABLE XVIII. REQUIREMENT-TO-TECHNOLOGY TRACEABILITY

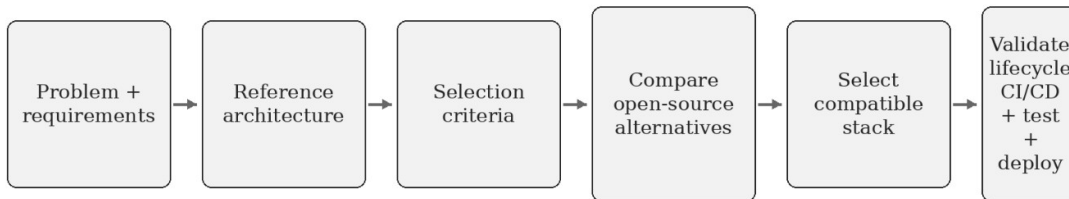
Assignment requirement	Section	Selected response	Evidence basis
Select Git platform	V	GitLab Community Edition	Research comparison + self-managed licensing documentation [4], [5], [9].
Recommend web framework	V	Django	Framework research + Django official documentation [6], [7], [10].
Choose appropriate database	VI	MySQL Community Edition	Database architecture study + MySQL documentation [8], [11].
Design deployment architecture	VI	Nginx + Gunicorn + Django + MySQL + Docker	Library-system study + Nginx/Docker documentation [3], [12], [13], [21].
Select testing tools	VII	pytest + Bruno + Selenium + JMeter + OWASP ZAP	Automated-testing and DevSecOps research + official project docs [14]–[17], [19], [20].
Justify every selection	IV–VIII	Requirements + criteria + trade-offs	Research synthesis, official documentation, and case-specific reasoning.
Comprehensive architecture outcome	V–IX, XII	Integrated lifecycle architecture	Cross-layer synthesis from source-control through maintenance.



CASE STUDY REPORT

Requirements-driven technology selection

Requirements → criteria → alternatives → compatible stack → lifecycle validation



Proposed open-source environment

